

To demonstrate originality, this document provides complete, step-by-step documentation and corresponding screenshots taken throughout the implementation of Technical Assessment 2.

TASK 1: SET UP THE ENVIRONMENT

- Write a function to **check if a room is dirty** and a function to **clean a room**.

```
18 class RoomStatus(str, Enum):
19     DIRTY = "Dirty"
20     CLEAN = "Clean"
21
22 # TASK 1: ENVIRONMENT SETUP & ABSTRACTION
23 class DualChamberEnvironment:
24     """Manages the two-room spatial state dictionary and mutation hooks."""
25
26     def __init__(self, room_a_state: str = "Clean", room_b_state: str = "Clean"):
27         # Task 1: Initialize two rooms using a Python dictionary
28         self._chambers: dict[str, RoomStatus] = {
29             'A': RoomStatus.DIRTY if room_a_state.strip().capitalize() == "Dirty" else RoomStatus.CLEAN,
30             'B': RoomStatus.DIRTY if room_b_state.strip().capitalize() == "Dirty" else RoomStatus.CLEAN
31         }
32
33     @property
34     def chambers(self) -> dict[str, str]:
35         return {room: status.value for room, status in self._chambers.items()}
36
37     def is_dirty(self, room: str) -> bool:
38         """Task 1: Sensor function to verify chamber dirt presence."""
39         return self._chambers.get(room) == RoomStatus.DIRTY
40
41     def clean_room(self, room: str) -> None:
42         """Task 1: Actuator hook to sanitize the designated room."""
43         if room in self._chambers:
44             self._chambers[room] = RoomStatus.CLEAN
45
```

TASK 2: IMPLEMENT THE RULE-BASED AGENT

```
46 # TASK 2: RULE-BASED REFLEX AGENT
47 class ReflexVacuumAgent:
48     """Production-rule reflex agent executing condition-action directives."""
49
50     def __init__(self, origin: str = 'A'):
51         self.location: str = origin.upper()
52
53     def move(self) -> None:
54         """Task 2: Navigates across the binary topological room layout."""
55         self.location = 'B' if self.location == 'A' else 'A'
56
57     def perceive_and_act(self, env: DualChamberEnvironment) -> tuple[str, str]:
58         """
59         Task 2: Evaluate sensory percept and trigger conditional action.
60         - Rule 1: If current chamber is Dirty -> Purge contaminant.
61         - Rule 2: If current chamber is Clean -> Relocate to counter-chamber.
62         """
63         origin = self.location
64
65         # Rule 1
66         if env.is_dirty(origin):
67             env.clean_room(origin)
68             return ("RULE_1", f"Sanitized dirty chamber [{origin}]")
69
70         # Rule 2
71         self.move()
72         return ("RULE_2", f"Relocated chamber: [{origin}] -> [{self.location}]")
```

TASK 3: RUN THE IMPLEMENTATION

```

74 # TASK 3: SIMULATION RUNNER & HUD PRINTER
75 def print_telemetry_card(step_num: int, origin_loc: str, rule_fired: str, action_desc: str, room_states: dict[str, str]) -> None:
76     """Formats and prints an ASCII telemetry HUD card for each execution step."""
77     p = TerminalPalette
78
79     # State color tags
80     def badge(status: str) -> str:
81         return f"{p.RED}DIRTY{p.RESET}" if status == "Dirty" else f"{p.GREEN}CLEAN{p.RESET}"
82
83     border = f"{p.DIM}+-----+{p.RESET}"
84     rule_tag = f"{p.YELLOW}[CLEAN]{p.RESET}" if rule_fired == "RULE_1" else f"{p.CYAN}[RELOCATE]{p.RESET}"
85
86     print(border)
87     print(f"| {p.BOLD}CYCLE {step_num:02d}{p.RESET} :: {rule_tag} {action_desc:<37} |")
88     print(f"| {p.DIM}Percept Position :{p.RESET} Chamber [{p.MAGENTA}{origin_loc}{p.RESET}] |")
89     print(f"| {p.DIM}Chamber Telemetry:{p.RESET} Room A -> {badge(room_states['A']):<18} Room B -> {badge(room_states['B']):<18} |")
90     print(border)
91
92
93 def run_standard_simulation() -> None:
94     p = TerminalPalette
95     while True:
96         print(f"\n{p.CYAN}|_____|{p.RESET}")
97         print(f"| {p.CYAN}|{p.RESET} {p.BOLD}TASKS 1, 2 & 3: 2-ROOM AGENT SIMULATION BENCHMARK{p.RESET} {p.CYAN}|{p.RESET}")
98         print(f"| {p.CYAN}|_____|{p.RESET}\n")
99
100         # Task 1 & 3: Interactive environment parameter setup
101         raw_a = input(f"{p.DIM}» Initial State for Chamber A (Clean/Dirty): {p.RESET}").strip()
102         raw_b = input(f"{p.DIM}» Initial State for Chamber B (Clean/Dirty): {p.RESET}").strip()
103         raw_loc = input(f"{p.DIM}» Initial Agent Deployment Chamber (A/B): {p.RESET}").strip().upper()
104         start_loc = raw_loc if raw_loc in ['A', 'B'] else 'A'
105
106         env = DualChamberEnvironment(raw_a, raw_b)
107         agent = ReflexVacuumAgent(start_loc)
108
109         try:
110             steps = int(input(f"{p.DIM}» Execution Step Limit: {p.RESET}"))
111         except ValueError:
112             steps = 4
113         print(f"{p.YELLOW}! Invalid integer; defaulting to 4 cycles.{p.RESET}")
114
115
116     print(f"\n{p.BOLD}--- EXECUTION TRACE INITIATED ---{p.RESET}\n")
117     time.sleep(0.2)
118
119     # Task 3: Execution loop
120     for step in range(1, steps + 1):
121         prior_loc = agent.location
122         rule_id, telemetry = agent.perceive_and_act(env)
123         print_telemetry_card(step, prior_loc, rule_id, telemetry, env.chambers)
124
125     # Interactive loop control
126     retry = input(f"\n{p.MAGENTA}» Rerun 2-Chamber simulation? (y/N): {p.RESET}").strip().lower()
127     if retry != 'y':
128         break

```

CS0065: INTELLIGENT SYSTEMS - REFLEX AGENT BENCHMARK

- [1] Execute Tasks 1, 2, & 3 (2-Chamber Production Model)
- [2] Execute Bonus Grid Matrix (3-Chamber Stochastic Model)
- [3] Terminate Session

» Command Interface Index (1/2/3): 1

TASKS 1, 2 & 3: 2-ROOM AGENT SIMULATION BENCHMARK

- » Initial State for Chamber A (Clean/Dirty): Clean
- » Initial State for Chamber B (Clean/Dirty): Clean
- » Initial Agent Deployment Chamber (A/B): A
- » Execution Step Limit: 5

--- EXECUTION TRACE INITIATED ---

```
+-----+
| CYCLE 01 :: [RELOCATE] Relocated chamber: [A] -> [B] |
| Percept Position : Chamber [A] |
| Chamber Telemetry: Room A -> CLEAN Room B -> CLEAN |
+-----+
+-----+
| CYCLE 02 :: [RELOCATE] Relocated chamber: [B] -> [A] |
| Percept Position : Chamber [B] |
| Chamber Telemetry: Room A -> CLEAN Room B -> CLEAN |
+-----+
+-----+
| CYCLE 03 :: [RELOCATE] Relocated chamber: [A] -> [B] |
| Percept Position : Chamber [A] |
| Chamber Telemetry: Room A -> CLEAN Room B -> CLEAN |
+-----+
+-----+
| CYCLE 04 :: [RELOCATE] Relocated chamber: [B] -> [A] |
| Percept Position : Chamber [B] |
| Chamber Telemetry: Room A -> CLEAN Room B -> CLEAN |
+-----+
+-----+
| CYCLE 05 :: [RELOCATE] Relocated chamber: [A] -> [B] |
| Percept Position : Chamber [A] |
| Chamber Telemetry: Room A -> CLEAN Room B -> CLEAN |
+-----+
```

» Rerun 2-Chamber simulation? (y/N): y

TASKS 1, 2 & 3: 2-ROOM AGENT SIMULATION BENCHMARK

- » Initial State for Chamber A (Clean/Dirty): Dirty
- » Initial State for Chamber B (Clean/Dirty): Dirty
- » Initial Agent Deployment Chamber (A/B): B
- » Execution Step Limit: 3

--- EXECUTION TRACE INITIATED ---

```
+-----+
| CYCLE 01 :: [CLEAN] Sanitized dirty chamber [B] |
| Percept Position : Chamber [B] |
| Chamber Telemetry: Room A -> DIRTY Room B -> CLEAN |
+-----+
| CYCLE 02 :: [RELOCATE] Relocated chamber: [B] -> [A] |
| Percept Position : Chamber [B] |
| Chamber Telemetry: Room A -> DIRTY Room B -> CLEAN |
+-----+
| CYCLE 03 :: [CLEAN] Sanitized dirty chamber [A] |
| Percept Position : Chamber [A] |
| Chamber Telemetry: Room A -> CLEAN Room B -> CLEAN |
+-----+
```

» Rerun 2-Chamber simulation? (y/N): █

CS0065: INTELLIGENT SYSTEMS - REFLEX AGENT BENCHMARK

- [1] Execute Tasks 1, 2, & 3 (2-Chamber Production Model)
- [2] Execute Bonus Grid Matrix (3-Chamber Stochastic Model)
- [3] Terminate Session

» Command Interface Index (1/2/3): 3

Session closed cleanly.

BONUS TASK: THIRD ROOM SIMULATION

```
129 # BONUS TASK: 3-CHAMBER VISUAL MATRIX & GRID ENGINE
130 class MultiChamberEnvironment:
131     """Dynamic spatial matrix with variable capacity (Rooms A, B, and C)."""
132
133     def __init__(self, states: dict[str, str]):
134         self.rooms = {k: RoomStatus.DIRTY if v.strip().capitalize() == "Dirty" else RoomStatus.CLEAN for k, v in states.items()}
135
136     def is_dirty(self, room: str) -> bool:
137         return self.rooms.get(room) == RoomStatus.DIRTY
138
139     def clean(self, room: str) -> None:
140         self.rooms[room] = RoomStatus.CLEAN
141
142     def dirty_nodes(self) -> list[str]:
143         return [node for node, state in self.rooms.items() if state == RoomStatus.DIRTY]
144
145
146 class StochasticMatrixAgent:
147     """Agent that handles dynamic chambers and targets dirty rooms randomly."""
148
149     def __init__(self, initial_node: str = 'A'):
150         self.node = initial_node.upper()
151
152     def cycle(self, env: MultiChamberEnvironment) -> str:
153         curr = self.node
154         if env.is_dirty(curr):
155             env.clean(curr)
156             return f"Purged contaminant at Room {curr}"
157
158         dirty_set = [r for r in env.dirty_nodes() if r != curr]
159         if dirty_set:
160             self.node = random.choice(dirty_set)
161             return f"Vector Lock: Shifted to Dirty Node [{self.node}]"
162
163         adjacent = [r for r in env.rooms.keys() if r != curr]
164         self.node = random.choice(adjacent)
165         return f"Patrol Vector: Sweeping Clean Node [{self.node}]"
166
167
168 def render_isometric_grid(room_dict: dict[str, RoomStatus], current_loc: str) -> None:
169     """Draws a clean, styled spatial grid showing the agent's live coordinates."""
```

```

168 def render_isometric_grid(room_dict: dict[str, RoomStatus], current_loc: str) -> None:
169     """Draws a clean, styled spatial grid showing the agent's live coordinates."""
170     p = TerminalPalette
171
172     def render_pod(chamber_id: str) -> tuple[str, str]:
173         is_here = (current_loc == chamber_id)
174         is_polluted = (room_dict[chamber_id] == RoomStatus.DIRTY)
175
176         status_txt = f"{p.RED}DIRTY{p.RESET}" if is_polluted else f"{p.GREEN}CLEAN{p.RESET}"
177         cursor_txt = f"{p.YELLOW}[AGENT]{p.RESET}" if is_here else " "
178
179         line_status = f"| Status : {status_txt:<15} "
180         line_cursor = f"| Pos      : {cursor_txt:<18} "
181         return line_status, line_cursor
182
183     box_top = "+-----+-----+-----+-----+"
184     box_header = f"| Chamber A          | Chamber B          | Chamber C          |"
185     box_div = "+-----+-----+-----+-----+"
186     box_bottom = "+-----+-----+-----+-----+"
187
188     s_a, c_a = render_pod('A')
189     s_b, c_b = render_pod('B')
190     s_c, c_c = render_pod('C')
191
192     print(f"{p.DIM}{box_top}{p.RESET}")
193     print(f"{p.BOLD}{box_header}{p.RESET}")
194     print(f"{p.DIM}{box_div}{p.RESET}")
195     print(f"{s_a}{s_b}{s_c}|")
196     print(f"{c_a}{c_b}{c_c}|")
197     print(f"{p.DIM}{box_bottom}{p.RESET}")
198
199
200 def run_bonus_simulation() -> None:
201     p = TerminalPalette
202     while True:
203         print(f"\n{p.MAGENTA}| {p.RESET}")
204         print(f"{p.MAGENTA}| {p.RESET} {p.BOLD}BONUS: 3-CHAMBER VISUAL MATRIX SIMULATION{p.RESET} {p.MAGENTA}| {p.RESET}")
205         print(f"{p.MAGENTA}| {p.RESET}\n")
206
207         states = {
208             'A': input(f"{p.DIM}» Chamber A State (Clean/Dirty): {p.RESET}"),
209             'B': input(f"{p.DIM}» Chamber B State (Clean/Dirty): {p.RESET}"),

```


CS0065: INTELLIGENT SYSTEMS - REFLEX AGENT BENCHMARK

- [1] Execute Tasks 1, 2, & 3 (2-Chamber Production Model)
- [2] Execute Bonus Grid Matrix (3-Chamber Stochastic Model)
- [3] Terminate Session

» Command Interface Index (1/2/3): 2

BONUS: 3-CHAMBER VISUAL MATRIX SIMULATION

- » Chamber A State (Clean/Dirty): Dirty
- » Chamber B State (Clean/Dirty): Dirty
- » Chamber C State (Clean/Dirty): Dirty
- » Deploy Agent to Chamber (A/B/C): C
- » Step Limit: 4

T-0 MATRIX INITIALIZATION:

Chamber A	Chamber B	Chamber C
Status : DIRTY	Status : DIRTY	Status : DIRTY
Pos :	Pos :	Pos : [AGENT]

STEP 01 ACTUATOR: Purged contaminant at Room C

Chamber A	Chamber B	Chamber C
Status : DIRTY	Status : DIRTY	Status : CLEAN
Pos :	Pos :	Pos : [AGENT]

STEP 02 ACTUATOR: Vector Lock: Shifted to Dirty Node [A]

Chamber A	Chamber B	Chamber C
Status : DIRTY	Status : DIRTY	Status : CLEAN
Pos : [AGENT]	Pos :	Pos :

STEP 03 ACTUATOR: Purged contaminant at Room A

Chamber A	Chamber B	Chamber C
Status : CLEAN	Status : DIRTY	Status : CLEAN
Pos : [AGENT]	Pos :	Pos :

STEP 04 ACTUATOR: Vector Lock: Shifted to Dirty Node [B]

Chamber A	Chamber B	Chamber C
Status : CLEAN	Status : DIRTY	Status : CLEAN
Pos :	Pos : [AGENT]	Pos :